24.3 Playwright 层实现

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~180k tokens，输出 ~6k tokens（本节）

上一节解析了 CDP 低级层——直接通过 WebSocket 控制浏览器。这一节上升到 Playwright 层，这是 Agent 与网页交互的主要接口：智能元素定位、页面快照、交互操作、状态管理。

24.3.1 Playwright 会话管理

连接复用

Playwright 通过 CDP 连接到 Chrome，但与 `withCdpSocket()` 的一次性连接不同，Playwright 需要**持久连接**来维护页面状态：

```
// src/browser/pw-session.ts — 连接缓存
let cached: ConnectedBrowser | null = null;
let connecting: Promise<ConnectedBrowser> | null = null;
```

通过 `cached` 和 `connecting` 实现连接复用——同一时刻只维护一个 Playwright 到 Chrome 的连接，所有操作共享这个连接。

页面状态追踪

每个 Page 对象关联一个 `PageState`，用 `WeakMap` 管理：

```
// src/browser/pw-session.ts — 页面状态
type PageState = {
  console: BrowserConsoleMessage[];  // 控制台消息（最多 500 条）
  errors: BrowserPageError[];        // 页面错误（最多 200 条）
  requests: BrowserNetworkRequest[]; // 网络请求（最多 500 条）
  requestIds: WeakMap<Request, string>;

  // 角色引用（Role Refs）——页面快照中元素的标识
  roleRefs?: Record<string, { role: string; name?: string; nth?: number }>;
  roleRefsMode?: "role" | "aria";
  roleRefsFrameSelector?: string;
};

const pageStates = new WeakMap<Page, PageState>();
```

> **衍生解释**： `WeakMap` 是 JavaScript 的一种特殊 Map，其键是弱引用——当键对象（这里是 Page）被垃圾回收时，对应的 entry 自动消失。这正好适合管理页面状态：当 Playwright 销毁 Page 对象后，相关的状态数据不会造成内存泄漏。

Role Refs 缓存

页面快照会生成**角色引用**（如 `e1` 、 `e2` ），Agent 通过这些引用来操作元素。为了保持引用稳定性（即使 Playwright 返回了不同的 Page 对象），还有一个 target 级别的缓存：

```
const roleRefsByTarget = new Map<string, RoleRefsCacheEntry>();
const MAX_ROLE_REFS_CACHE = 50;
```

缓存键是 `${cdpUrl}::${targetId}` ，最多存储 50 个 target 的引用，FIFO 淘汰。

24.3.2 角色快照（Role Snapshot）

角色快照是 OpenClaw 浏览器控制的**核心创新**——它将网页的可访问性树转换为 Agent 可读的文本表示，并为每个交互元素分配一个短引用 ID。

三类 ARIA 角色

```
// src/browser/pw-role-snapshot.ts — 角色分类
const INTERACTIVE_ROLES = new Set([
  "button", "link", "textbox", "checkbox", "radio",
  "combobox", "listbox", "menuitem", "option",
  "searchbox", "slider", "spinbutton", "switch", "tab", "treeitem",
]);

const CONTENT_ROLES = new Set([
  "heading", "cell", "gridcell", "columnheader", "rowheader",
  "listitem", "article", "region", "main", "navigation",
]);

const STRUCTURAL_ROLES = new Set([
  "generic", "group", "list", "table", "row", "rowgroup",
  "grid", "treegrid", "menu", "menubar", "toolbar", "tablist",
  "tree", "directory", "document", "application",
]);
```

> **衍生解释**：ARIA（Accessible Rich Internet Applications）是 W3C 制定的标准，定义了 Web 元素的**语义角色**（Role）。例如一个 `<div>` 可以有 `role="button"` 表示它是按钮，`<input type="text">` 自动获得 `role="textbox"`。屏幕阅读器（如 VoiceOver、NVDA）依赖这些角色来向视障用户描述页面结构。OpenClaw 复用了这个标准——Agent 通过角色来"看"页面，就像屏幕阅读器一样。

快照格式

角色快照的输出类似这样：

```
- navigation "Main Menu"
  - link "Home" [e1]
  - link "Products" [e2]
  - button "Cart (3)" [e3]
- main
  - heading "Welcome to Store" [level=1]
  - region "Featured"
    - link "Product A" [e4]
    - button "Add to Cart" [e5]
    - link "Product B" [e6]
    - button "Add to Cart" [e7]
  - textbox "Search..." [e8]
  - button "Search" [e9]
```

方括号中的 `[e1]` 、 `[e2]` 等就是**角色引用**——Agent 在后续操作中使用 `e3` 来点击 "Cart" 按钮。

引用分配与去重

```typescript
// src/browser/pw-role-snapshot.ts — 引用追踪器
function createRoleNameTracker(): RoleNameTracker {
  const counts = new Map<string, number>();
  const refsByKey = new Map<string, string[]>();
  return {
    getKey(role, name) { return `${role}:${name ?? ""}`; },
    getNextIndex(role, name) {
      const key = this.getKey(role, name);
      const current = counts.get(key) ?? 0;
      counts.set(key, current + 1);
      return current;
    },
    trackRef(role, name, ref) {
      const key = this.getKey(role, name);
      const refs = refsByKey.get(key) ?? [];
      refs.push(ref);
      refsByKey.set(key, refs);
    },
    getDuplicateKeys() { /* 找出同名同角色的重复元素 */ },
  };
}
```

当页面上有多个同名按钮（如两个 "Add to Cart"），追踪器会为它们分配不同的 `nth` 索引，确保引用不歧义。

快照统计

```
export function getRoleSnapshotStats(snapshot: string, refs: RoleRefMap) {
  const interactive = Object.values(refs)
    .filter(r => INTERACTIVE_ROLES.has(r.role)).length;
  return {
    lines: snapshot.split("\n").length,
    chars: snapshot.length,
    refs: Object.keys(refs).length,
    interactive,  // 可交互元素数量
  };
}
```

24.3.3 三种快照模式

OpenClaw 提供三种页面快照方式，各有侧重：

1. AI Snapshot（推荐）

```
// src/browser/pw-tools-core.snapshot.ts — snapshotAiViaPlaywright
export async function snapshotAiViaPlaywright(opts) {
  const page = await getPageForTargetId(opts);
  const maybe = page as WithSnapshotForAI;

  // 使用 Playwright 内置的 _snapshotForAI
  const result = await maybe._snapshotForAI({
    timeout: opts.timeoutMs ?? 5000,
    track: "response",
  });

  let snapshot = String(result?.full ?? "");
  // 截断过大的快照
  if (limit && snapshot.length > limit) {
    snapshot = `${snapshot.slice(0, limit)}\n\n[...TRUNCATED — page too large]`;
  }

  // 构建角色引用
  const built = buildRoleSnapshotFromAiSnapshot(snapshot);
  storeRoleRefsForTarget({ page, refs: built.refs, mode: "aria" });

  return { snapshot, refs: built.refs };
}
```

`_snapshotForAI` 是 Playwright 内部的一个方法（前缀下划线表示非公开 API），它返回一个为 AI 优化的页面文本表示。

2. ARIA Snapshot

```
// 通过 CDP Accessibility.getFullAXTree 获取完整可访问性树
export async function snapshotAriaViaPlaywright(opts) {
  const session = await page.context().newCDPSession(page);
  const res = await session.send("Accessibility.getFullAXTree");
  const nodes = formatAriaSnapshot(res.nodes, limit);
  return { nodes };
}
```

直接获取 Chrome 的可访问性树——信息最完整但体积较大。

3. Role Snapshot

```
// 基于 ARIA 快照构建角色树
export async function snapshotRoleViaPlaywright(opts) {
  const page = await getPageForTargetId(opts);
  const session = await page.context().newCDPSession(page);
  const res = await session.send("Accessibility.getFullAXTree");

  const built = buildRoleSnapshotFromAriaSnapshot(res.nodes, {
    interactive: opts.interactive,
    maxDepth: opts.maxDepth,
    compact: opts.compact,
  });

  storeRoleRefsForTarget({ page, refs: built.refs, mode: "role" });
  return { snapshot: built.snapshot, refs: built.refs, stats: built.stats };
}
```

Role Snapshot 支持过滤选项——可以只显示交互元素、限制深度、压缩结构性节点——适合 Agent 快速了解页面可操作元素。

24.3.4 Playwright 工具核心

`pw-tools-core.ts` 是一个桶文件，聚合了八个子模块：

```
// src/browser/pw-tools-core.ts
export * from "./pw-tools-core.activity.js";
export * from "./pw-tools-core.downloads.js";
export * from "./pw-tools-core.interactions.js";
export * from "./pw-tools-core.responses.js";
export * from "./pw-tools-core.snapshot.js";
export * from "./pw-tools-core.state.js";
export * from "./pw-tools-core.storage.js";
export * from "./pw-tools-core.trace.js";
```

interactions — 交互操作

核心交互操作都通过**角色引用**定位元素：

```typescript
// src/browser/pw-tools-core.interactions.ts — click（简化）
export async function clickViaPlaywright(opts: {
  cdpUrl: string;
  targetId?: string;
  ref: string;              // 角色引用，如 "e3"
  doubleClick?: boolean;
  button?: "left" | "right" | "middle";
  modifiers?: Array<"Alt" | "Control" | "Meta" | "Shift">;
  timeoutMs?: number;
}) {
  const page = await getPageForTargetId(opts);
  ensurePageState(page);
  restoreRoleRefsForTarget({ cdpUrl: opts.cdpUrl, targetId: opts.targetId, page });

  const ref = requireRef(opts.ref);
  const locator = refLocator(page, ref);  // 通过引用获取 Playwright Locator

  if (opts.doubleClick) {
    await locator.dblclick({ timeout, button: opts.button, modifiers: opts.modifiers })
  } else {
    await locator.click({ timeout, button: opts.button, modifiers: opts.modifiers });
  }
}
```

`refLocator(page, ref)` 是关键桥梁——它在 `PageState.roleRefs` 中查找引用 `e3` 对应的 `{ role: "button", name: "Cart (3)" }`，然后构造 Playwright 的 `page.getByRole("button", { name: "Cart (3)" })` 定位器。

完整的交互操作集：

函数	用途
`clickViaPlaywright`	点击/双击元素
`hoverViaPlaywright`	悬停
`typeViaPlaywright`	在输入框中打字
`fillFormViaPlaywright`	批量填写表单
`selectOptionViaPlaywright`	选择下拉选项
`pressKeyViaPlaywright`	按键（如 Enter、Tab）

函数	用途
dragViaPlaywright	拖拽
scrollIntoViewViaPlaywright	滚动到元素可见
highlightViaPlaywright	高亮元素（调试用）
setInputFilesViaPlaywright	文件上传
armFileUploadViaPlaywright	预备文件选择器
armDialogViaPlaywright	预备对话框处理

downloads — 下载管理

```
export async function downloadViaPlaywright(opts) { /* 导航到 URL 触发下载 */ }
export async function waitForDownloadViaPlaywright(opts) { /* 等待下载完成 */ }
```

state — 页面状态

```
export async function navigateViaPlaywright(opts) { /* 导航到 URL */ }
export async function evaluateViaPlaywright(opts) { /* 执行 JS */ }
export async function resizeViewportViaPlaywright(opts) { /* 调整视口大小 */ }
export async function setDeviceViaPlaywright(opts) { /* 模拟设备 */ }
export async function setGeolocationViaPlaywright(opts) { /* 设置地理位置 */ }
export async function emulateMediaViaPlaywright(opts) { /* 模拟媒体类型 */ }
```

storage — 存储管理

```
export async function cookiesGetViaPlaywright(opts) { /* 读取 cookie */ }
export async function cookiesSetViaPlaywright(opts) { /* 设置 cookie */ }
export async function cookiesClearViaPlaywright(opts) { /* 清除 cookie */ }
export async function storageGetViaPlaywright(opts) { /* 读取 localStorage */ }
export async function storageSetViaPlaywright(opts) { /* 设置 localStorage */ }
export async function storageClearViaPlaywright(opts) { /* 清除 localStorage */ }
```

trace — 性能追踪

```
export async function traceStartViaPlaywright(opts) { /* 开始录制 Playwright Trace */ }
export async function traceStopViaPlaywright(opts) { /* 停止并保存 Trace 文件 */ }
```

24.3.5 AI 辅助模块聚合

`pw-ai.ts` 将所有 Playwright 操作重新导出为统一的 API 表面：

```
// src/browser/pw-ai.ts — 60+ 个函数重导出
export {
  // 会话管理
  getPageForTargetId, ensurePageState, closePlaywrightBrowserConnection,
  listPagesViaPlaywright, createPageViaPlaywright, refLocator,

  // 交互操作
  clickViaPlaywright, hoverViaPlaywright, fillFormViaPlaywright,
  typeViaPlaywright, pressKeyViaPlaywright, dragViaPlaywright,

  // 快照
  snapshotAiViaPlaywright, snapshotAriaViaPlaywright, snapshotRoleViaPlaywright,
  takeScreenshotViaPlaywright, screenshotWithLabelsViaPlaywright,

  // 状态
  navigateViaPlaywright, evaluateViaPlaywright, resizeViewportViaPlaywright,

  // 存储
  cookiesGetViaPlaywright, storageGetViaPlaywright,

  // 网络
  getConsoleMessagesViaPlaywright, getNetworkRequestsViaPlaywright,

  // ... 等 60+ 个函数
} from "./pw-tools-core.js";
```

本节小结

1. **Playwright 连接复用**——全局缓存一个到 Chrome 的持久连接，所有操作共享
2. **页面状态用 WeakMap 管理**——控制台消息、网络请求、页面错误各有容量上限（500/500/200）
3. **角色快照是 Agent "看"网页的方式**——将可访问性树转为缩进文本，为每个交互元素分配短引用 ID（如 `e1`）
4. **三种快照模式**：AI Snapshot（Playwright 内建优化）、ARIA Snapshot（完整可访问性树）、Role Snapshot（可过滤/压缩）
5. **所有交互操作通过角色引用定位**——Agent 看到 `[e3]` 就用 `ref: "e3"` 来点击，底层通过 `getByRole()` 解析
6. **工具核心分为八大模块**：交互、快照、下载、响应、活动、状态、存储、追踪
7. **`pw-ai.ts` 聚合了 60+ 个 Playwright 操作函数**，为浏览器服务器提供统一的 API 表面